

Table of Contents

Audit technique et architectural du projet Claviger

Version auditee : branche main fournie le 7 septembre 2026

Perimetre : code source Python, configuration, persistance SQLite, commandes et UI Discord, reporting, tests automatises et README mis a jour fourni separement.

Cible : V1 de gestion de roles Discord, avec deploiement Docker sur mini-PC prevu ulterieurement.

Limite de verification. Les 252 fonctions de test annoncees dans le README ont bien ete retrouvees statiquement dans la suite. Le code Python compile avec compileall. L'execution complete de pytest n'a pas pu etre reproduite dans l'environnement d'audit car discord.py et aiosqlite n'y etaient pas installes et l'acces reseau permettant de les installer etait indisponible. Ce point n'est donc pas traite comme un echec du projet.

1. Resume executif

1.1 Verdict general

Claviger est une V1 techniquement credible. Le projet a depasse le stade du bot Discord monolithique : la logique de reconciliation est separee entre construction du questionnaire, planification des changements, execution, orchestration de workflow et persistance. Cette separation est visible dans le code, pas seulement dans le README.

Le projet presente une qualite nettement superieure a un bot personnel construit autour de handlers Discord contenant directement des appels SQL et des mutations de roles. Les planners sont testables sans Discord, les repositories encapsulent SQLite, les synchronisations de catalogues disposent d'une transaction globale, les operations de roles sont pre-validees avant la premiere mutation et la politique par serveur applique un default restrictif pour les serveurs non configures.

La faiblesse principale n'est plus la logique metier. Elle se situe desormais dans l'exploitation : gestion d'erreurs inegale, journalisation partiellement branchee, absence de configuration runtime du logging, perte des stack traces dans plusieurs erreurs reportees, absence de strategie explicite pour les mutations Discord partiellement appliquees et quelques incoherences entre les controles de securite effectues par des services differents.

La recommandation n'est **pas** de reecrire le projet. Un refactoring incremental est adapte.

1.2 Appreciation par axe

Axe	Appreciation	Commentaire
Qualite Backend	Solide pour une V1	Repositories, services, models, orchestration et transactions sont reels et distincts.
Architecture	Bonne base	Separation utile, mais le package services melange encore logique applicative et integration Discord.
Modularite	En progression	Plusieurs composants sont reutilisables; d'autres gardent des conventions Succumbrae/Noctis ou des dependances Discord directes.
Maintenabilite	Bonne avec points chauds	Les grands fichiers restent concentres (role_channel_catalog_repository.py, database_command.py, bot.py). Quelques duplications peuvent diverger.
Testabilite	Forte	252 tests detectes, planners purs, nombreux mocks et tests SQLite reels.
Preparation production	Insuffisante en l'etat	Le coeur fonctionne, mais l'observabilite, le verrouillage des dependances et les artefacts de deploiement ne sont pas encore finalises.
Capacite a accueillir de l'IA	Bonne fondation	Le systeme de capabilities, les services et la separation des actions sont utiles, mais les futures actions sensibles devront avoir une autorisation

Axe	Appreciation	Commentaire
		explicite et auditables.

1.3 Forces principales

1. **Reconciliation plutot qu'attribution imperative.** Les workflows reconstruisent l'etat courant puis produisent un plan de changements. Cela facilite l'idempotence et limite les effets de stale UI.
2. **Preflight des roles avant mutation.** MemberRoleExecutorService et NoctisRoleExecutorService resolvent et valident tous les roles prevus avant la premiere modification Discord (member_role_executor_service.py:54-78, noctis_role_executor_service.py:54-75).
3. **Transactions SQLite serieuses.** Les migrations utilisent BEGIN IMMEDIATE, commit et rollback; la synchronisation de tous les catalogues est appliquee dans une transaction commune.
4. **Identite stable pour les catalogues.** Les entrees sont rattachees au role_id Discord et conservent les metadonnees humaines lors des rafraichissements.
5. **Reporting extensible deja amorce.** ReportService diffuse un ReportEvent vers une liste de Reporter, ce qui permet console/logging, forum Discord et futurs sinks sans coupler le metier a une destination.
6. **Tests importants et orientes comportements.** La suite couvre la planification, l'execution, les migrations, les repositories, les policies, les commandes et les UI.
7. **Least privilege partiellement bien applique.** Les commandes administratives verifient le proprietaire du serveur; les serveurs non configures recoivent une policy restrictive.
8. **SQL correctement parametrise.** Les valeurs dynamiques passent par des parametres, et les noms de table/colonne internes sont verifies par une expression reguliere avant interpolation.

1.4 Faiblesses principales

1. Les erreurs des parcours publics /membre et des modales sont parfois absorbees sans report exploitable.
2. Le reporting d'erreur conserve souvent uniquement str(error) : le traceback d'origine est perdu.
3. PythonLoggingReporter produit du texte semi-structure, mais aucun logging.basicConfig/dictConfig n'est present; les evenements INFO ne sont donc pas garantis d'etre visibles en production.
4. Une mutation Discord peut echouer apres plusieurs suppressions/ajouts et laisser un etat partiel sans objet d'erreur decrivant exactement ce qui a deja ete applique.
5. RoleChannelDiscoveryService marque un role comme gerable uniquement selon sa position, contrairement aux executors et a RoleDiscoveryService, qui excluent aussi les roles managed et @everyone.
6. Les roles fixes et les salons de workflow sont identifies par nom, ce qui reste ambigu sur Discord en cas de doublons.

7. Aucune contrainte ne garantit l'unicite logique de interest_key/access_key dans un serveur.
8. Plusieurs duplications et incoherences de nomenclature temoignent d'un refactoring encore inacheve, notamment la copie de commands/roles.py dans commands/__init__.py.
9. Le composition root ClavigerBot.__init__ reste lisible aujourd'hui mais deviendra difficile a faire evoluer si les futurs modules IA, moderation et Web sont ajoutes au meme endroit.
10. Le deploiement Docker et la reproductibilite exacte des dependances ne sont pas encore presents dans la branche auditee.

2. Cartographie du projet

2.1 Arborescence synthetique

src/claviger/

```
| - main.py          # point d'entree CLI
| - bot.py           # composition root + enregistrement des commandes
| - config.py        # variables d'environnement
| - commands/        # adaptateurs Discord slash commands
| - ui/              # modales et vues Discord
| - services/        # logique applicative + certains adaptateurs Discord
| - models/          # dataclasses de plan, resultat, catalogues, snapshots
| - repositories/    # persistance SQLite
| - database/        # connexion, schema, migrations, statut
| - policies/        # politique effective par serveur
| - reporting/       # evenements et destinations de reporting
|- constants/        # constantes metier/serveur
```

tests/

```
| - services/
| - repository/
| - database/
| - commands/
| - ui/
| - reporting/
| - test_claviger/   # tests historiques issus du decoupage de l'ancien module
|- plusieurs tests racine encore a reorganiser
```

2.2 Principaux composants

Zone	Responsabilite actuelle	Evaluation
main.py	Charger le token et lancer le bot	Simple et correct.

Zone	Responsabilite actuelle	Evaluation
bot.py	Construire les dependances et enregistrer les commandes	Bon composition root, mais volumineux a terme.
commands/	Controle du contexte Discord, autorisation, orchestration UI	Correct comme couche d'adaptation; gestion d'erreur dupliquee.
ui/	Collecter des choix et soumettre aux coordinators	Bonne protection actor_id; reporting incomplet.
services/	Planners, executors, workflows, discovery, authorization	Coeur le plus important; qualite heterogene mais globalement bonne.
repositories/	CRUD et synchronisation des catalogues	Solide; base generique assez volumineuse.
database/	Connexion et migrations explicites	Bonne base pour SQLite mono-instance.
policies/	Defaults, overrides et resolution par guild	Bonne idee; le fallback Succumbrae doit etre isole.
reporting/	Evenement structure et multi-destination	Tres bon point d'extension, mais pas encore industrialise.

2.3 Flux /membre

Utilisateur Discord

- > member_command
- > controle DB + policy + salon
- > MemberWorkflowCoordinatorService.build_questionnaire
- > MemberInterestQuestionnaireService
- > InterestCatalogRepository -> SQLite
- > MemberQuestionnaireModal

Soumission

- > MemberWorkflowCoordinatorService.apply_selection
- > reconstruction de l'etat courant
- > MemberRolePlannerService (pur)
- > MemberRoleExecutorService
- > RoleManager
- > API Discord

Point positif important : la soumission ne fait pas confiance au questionnaire construit plusieurs minutes auparavant. L'état du catalogue et les rôles courants sont reconstruits avant de calculer le plan (member_workflow_coordinator_service.py).

2.4 Flux /noctis

Utilisateur Discord

- > noctis_command
- > controle DB + policy + salon
- > NoctisWorkflowCoordinatorService
- > AdultAccessQuestionnaireService
- > AccessCatalogRepository -> SQLite
- > AdultAccessWorkflowService (themes base/IA)
- > NoctisQuestionnaireModal

Soumission

- > reconstruction et validation
- > NoctisRolePlannerService
- > NoctisRoleExecutorService
- > RoleManager
- > API Discord

La préférence IA est additive et le planner distingue les rôles gérés par le workflow des rôles extérieurs.

2.5 Flux de synchronisation des catalogues

/claviger catalog sync

- > RoleChannelDiscoveryService : snapshot Discord unique
- > CatalogRegistry : définitions actives
- > CatalogSyncPlanner : plan pur par catalogue
- > BEGIN IMMEDIATE
- > repositories.apply_sync_plan(...)
- > COMMIT global
- > rollback global si un catalogue échoue

C'est l'un des flux les mieux conçus du projet : observation, planification puis application transactionnelle sont nettement séparées.

2.6 Flux de reporting

Code appelant

- > ReportEvent
- > ReportService
 - | - PythonLoggingReporter
 - ` - DiscordForumReporter (optionnel)

Le design est bon, mais son integration est incomplete : certains handlers emettent des evenements, d'autres absorbent l'erreur, et le sink Python n'est pas configure au niveau application.

3. Analyse architecturale

3.1 Separation des responsabilites

La separation questionnaire -> planner -> executor -> coordinator est techniquement justifiee. Le planner peut etre teste sans acces Discord, l'executor porte les contraintes d'application, et le coordinator gere le workflow sans mettre toute la logique dans les slash commands.

Ce decoupage est plus pertinent qu'une Clean Architecture complete pour la taille actuelle. Il resout des problemes concrets : stale state, validation des selections, idempotence et testabilite.

La limite actuelle est que le repertoire services/ contient deux familles differentes :

- des services quasi-purs (MemberRolePlannerService, CatalogSyncPlanner, AdultAccessWorkflowService);
- des adaptateurs directement dependants de discord.py (RoleManager, RoleDiscoveryService, executors, questionnaire services).

Ce melange n'est pas bloquant. Il deviendra en revanche couteux si les memes fonctions doivent etre invoquees depuis une interface Web ou un agent IA. Le prochain refactoring doit isoler les actions Discord derriere quelques contrats explicites, pas introduire une couche d'abstraction pour chaque classe.

3.2 Couplage et injection de dependances

ClavigerBot.__init__ construit explicitement les services et repositories. Cette approche est preferable a des singletons ou imports globaux caches : les dependances sont visibles et les tests peuvent remplacer les collaborateurs.

Le revers est un constructeur de plus de 170 lignes de wiring (bot.py:93-263). Il s'agit d'un composition root acceptable aujourd'hui, mais il ne doit pas devenir un service locator contenant toute la future plateforme.

Recommandation incrementale : creer, lorsque le prochain sous-systeme important apparait, de petites factories/modules de composition par domaine (build_member_module, build_catalog_module, etc.) qui retournent les commandes ou les coordinators necessaires. Ne pas introduire de framework DI.

3.3 Cohesion

Les planners ont une bonne cohesion. Les coordinators sont egalement raisonnables.

Deux points chauds meritent surveillance :

- RoleChannelCatalogRepository (667 lignes) concentre lecture, creation, refresh, update de metadata, application de plan et mapping SQL. Ce volume est encore justifie par la reutilisation entre deux catalogues, mais toute nouvelle responsabilite doit plutot aller dans un composant separe.
- database_command.py (359 lignes) regroupe formatage, status, initialize et migrate. Ce n'est pas dangereux, mais les commandes administratives pourraient etre separees si d'autres operations arrivent.

3.4 Persistence et transactions

La couche SQLite est un point fort :

- le fichier n'est pas cree implicitement lors des simples lectures;
- PRAGMA foreign_keys = ON est active par connexion;
- les migrations sont versionnees par PRAGMA user_version;
- chaque migration utilise une transaction;
- la synchronisation multi-catalogue utilise une transaction englobante;
- les valeurs SQL sont parametrees;
- les identifiants SQL interpolables sont valides par _IDENTIFIER_PATTERN.

Aucune injection SQL exploitable n'a ete identifiee dans le code audite.

La prochaine evolution devrait ajouter des contraintes d'integrite metier (unicite des cles logiques) plutot que complexifier le moteur de persistance.

3.5 Gestion de l'etat Discord

Les workflows reconstruisent l'etat au moment de la soumission, ce qui reduit les problemes de modal obsolete. C'est un choix robuste.

En revanche, l'API Discord ne fournit pas de transaction multi-appels. La prevalidation empeche un plan invalide d'effectuer une premiere mutation, mais elle ne garantit pas l'atomicite une fois les appels reseau commences. Cette distinction doit etre explicitement documentee.

3.6 Policies et multi-guild

La plupart des donnees sont deja scoping par guild_id, et la GuildPolicy permet des overrides. Cela prepare correctement un futur multi-serveurs.

Le runtime est cependant encore mono-guild : les commandes sont enregistrees uniquement sur DISCORD_GUILD_ID, et le bootstrap repose sur un fallback_guild_id. C'est coherent avec la V1 et ne doit pas etre presente comme du multi-guild deja termine.

3.7 Preparation a l'IA

Aucun LLM ou agent n'est present, ce qui est coherent avec le perimetre V1.

Les briques utiles existent déjà : services, capabilities, plans d'action, reporting et tests. Pour une future couche agent, les services les plus sensibles devront être exposés via des outils explicites disposant de :

- schema d'entrée valide;
- capability requise;
- actor/context obligatoires;
- mode lecture vs mutation;
- resultat structure;
- journal d'audit;
- confirmation humaine pour les actions destructrices lorsque nécessaire.

Le LLM ne doit jamais obtenir un accès direct à `discord.Member.add_roles`, à SQLite ou à une commande d'administration.

4. Ce qui est bien conçu

4.1 Le planner de rôles est une vraie couche métier

`MemberRolePlannerService` et `NoctisRolePlannerService` ne modifient pas Discord. Ils valident les clés et calculent `add_role_ids/remove_role_ids`. Cette séparation est facile à expliquer en entretien et produit des tests précis.

4.2 Les exécuteurs effectuent un préflight complet

Tous les rôles du plan sont résolus avant la première mutation, puis vérifiés contre la hiérarchie et le flag `managed`. Cela évite qu'une sélection contenant un rôle déjà supprimé ou inaccessible commence à modifier le membre avant de découvrir le problème.

4.3 Le catalogue sépare identité technique et métadonnées utilisateur

Les synchronisations rafraîchissent les données possédées par Discord sans écraser `label`, `description` et `emoji`. C'est un bon exemple de séparation entre source de vérité externe et enrichissement applicatif.

4.4 La synchronisation est planifiée avant écriture

Le `CatalogSyncPlanner` reçoit un snapshot et l'état persiste, puis retourne un `CatalogSyncPlan`. Le coordinator applique ensuite tous les plans dans une transaction. Ce choix simplifie les tests et limite les états partiels en base.

4.5 Les serveurs inconnus ont un défaut restrictif

`SAFE_DEFAULT_POLICY` désactive la gestion de rôles et l'accès adulte. C'est conforme au principe du moindre privilège.

4.6 ReportService isole les destinations

Le Protocol Reporter et la politique "un reporter en panne ne bloque pas les autres" sont de bons choix. Le except Exception du ReportService est ici justifié : il constitue une frontière d'infrastructure dont le but explicite est d'isoler les sinks.

4.7 Les tests ciblent des invariants importants

Exemples : rollback global du catalog sync, absence de création implicite de DB, preflight avant mutation, préservation des métadonnées, refus des rôles non gérables, restauration des sélections et contrôle de l'acteur des modales.

5. Problemes architecturaux et recommandations

5.1 Gestion des erreurs inégale et silencieuse

Constat. Plusieurs points d'entrée publics font except Exception puis renvoient seulement un message utilisateur. Exemples : `member_command.py:38-65`, `member_questionnaire_modal.py:79-95`, `noctis_questionnaire_modal.py:101-119`, `catalog_metadata_modal.py` et `catalog_next_view.py`.

Impact. Un incident réel peut être invisible dans les logs et impossible à diagnostiquer après coup.

Recommandation. Conserver un message utilisateur neutre, mais brancher tous les boundaries Discord sur un mécanisme commun qui :

1. classe les exceptions attendues vs inattendues;
2. journalise une trace complète localement;
3. émet un ReportEvent sanitize si l'incident mérite un signal administratif;
4. associe `interaction.id`, `guild_id`, `actor_id`, `operation` et `duree`.

Priorité. P0 avant production.

5.2 Les tracebacks sont perdus dans les reports d'erreur

Constat. Les commandes attrapent souvent Exception as error puis stockent uniquement `details=str(error)`. PythonLoggingReporter appelle ensuite `logger.log` sans `exc_info`.

Impact. Le message d'erreur est conservé, mais pas la pile d'appels ayant conduit à l'incident. Deux erreurs identiques en surface deviennent difficiles à différencier.

Recommandation. Le sink local doit recevoir le traceback complet au point de capture. Le forum Discord doit recevoir une version courte et rédigée, pas la stack trace.

Priorité. P0.

5.3 Le logger Python n'est pas configure au runtime

Constat. Le projet instancie `logging.getLogger(...)`, mais aucune configuration centrale du logging n'est presente; `bot.py` utilise encore `print` dans `setup_hook` et `on_ready` (`bot.py:265-281`).

Impact. Les INFO peuvent etre ignores et le format depend de l'environnement. En Docker, la qualite des logs devient imprevisible.

Recommandation. Configurer le logging une fois au demarrage : `stdout`, `timestamp UTC`, `level`, `logger/module`, `operation` et `champs contextuels`. Remplacer les `print` par le `logger`.

Priorite. P0.

5.4 Les mutations Discord ne sont pas transactionnelles

Constat. Les executors pre-valident tous les roles, puis effectuent plusieurs appels `remove_role` et `add_role`. Une erreur reseau ou Discord peut survenir apres certaines mutations.

Impact. L'etat final peut etre partiel. Le resultat retourne ne sera jamais construit si une exception interrompt l'execution, donc le caller ignore precisement ce qui a deja change.

Recommandation. Introduire un resultat/exception de type "partial execution" contenant les role IDs deja ajoutes/retires, le role ayant echoue et l'operation. Ajouter une strategie de compensation best-effort uniquement si elle est sure. Ne pas promettre une atomicite que Discord ne fournit pas.

Priorite. P0 pour le reporting de l'etat partiel; P1 pour une compensation avancee.

5.5 Incoherence sur `role_manageable`

Constat. `RoleChannelDiscoveryService` utilise seulement `role < bot_member.top_role` (`role_channel_discovery_service.py:44-55`). Les executors rejettent en plus `role.managed`, et `RoleDiscoveryService` exclut les roles `managed/default`.

Impact. Le catalogue peut marquer une entree utilisable alors que l'executor la rejettera ensuite.

Recommandation. Centraliser un unique predicat de manageabilite reutilise par `discovery`, `catalogue` et `executor`. Ajouter un test avec `role.managed=True`.

Priorite. P0.

5.6 Roles et salons fixes identifies par nom

Constat. `member_role_name`, `adult_role_name`, `salutations_channel_name` et `adult_rules_channel_name` sont des noms. Discord autorise des noms de roles en doublon et des noms de salons identiques dans des categories differentes.

Impact. discord.utils.get(..., name=...) ou une comparaison de nom peut choisir/accepter une ressource ambiguë. Le scan admin detecte certains doublons, mais les workflows ne bloquent pas tous ces cas.

Recommandation. Court terme : exiger exactement un role portant le nom configure et echouer en cas de doublon. Moyen terme : persister des IDs stables pour les roles et salons fixes, avec les noms comme metadata d'affichage.

Priorite. P1, ou P0 si le serveur autorise des administrateurs non maitrises a creer des doublons.

5.7 Cles logiques de catalogue non uniques

Constat. Les tables utilisent (guild_id, role_id) comme PK, sans contrainte d'unicite sur (guild_id, interest_key) ou (guild_id, access_key). Le planner n'empêche pas deux roles Discord d'aboutir a la meme cle logique.

Impact. Un questionnaire peut contenir deux options partageant la meme valeur; un choix logique peut alors cibler plusieurs roles sans que l'utilisateur puisse les distinguer.

Recommandation. Detecter les doublons pendant catalog sync et les declarer invalides. Si la regle metier est bien "une cle logique = un role par guild", ajouter ensuite une contrainte UNIQUE via migration.

Priorite. P1.

5.8 Duplication quasi exacte des executors et models de plan/resultat

Constat. MemberRoleExecutorService et NoctisRoleExecutorService sont presque identiques. Les MemberRolePlan/NoctisRolePlan et resultats portent egalement la meme structure avec de petites divergences de defaults/slots/proprietes.

Impact. Les correctifs de securite ou de gestion d'erreur doivent etre appliques deux fois. La divergence actuelle sur les petits details montre deja ce risque.

Recommandation. Extraire un RoleMutationPlan, RoleMutationResult et un executor generique. Conserver les planners et coordinators metier specifiques.

Priorite. P2 apres stabilisation de la V1.

5.9 Duplication de la regle is_publicly_ready

Constat. MemberInterestQuestionnaireService._is_publicly_available recalcule pratiquement la propriete RoleChannelCatalogEntry.is_publicly_ready, tandis que le workflow adulte utilise directement cette propriete.

Impact. Une evolution des criteres peut diverger entre les deux workflows.

Recommandation. Utiliser une seule source de verite.

Priorite. P2.

5.10 Duplication morte dans `commands/_init_.py`

Constat. `commands/_init_.py` et `commands/roles.py` contiennent la meme implementation de `create_roles_group`, alors que l'application importe celle de `roles.py`.

Impact. Dette de refactoring et risque qu'un futur developpeur modifie la mauvaise copie.

Recommandation. Vider l'initializer ou n'y laisser que des exports explicites; conserver l'implementation dans un fichier unique.

Priorite. P1/P2; faible risque runtime, correction simple.

5.11 Fallback Succumbrae active en cas de DB indisponible

Constat. `PolicyResolver` renvoie `SUCCUMBRAE_FALLBACK_POLICY`, dont les fonctions de roles sont activees, lorsque la DB est indisponible pour le guild principal. Les commandes de mutation actuelles verifient heureusement l'etat DB avant d'arriver a ce point.

Impact. Le comportement actuel reste protege, mais cette policy constitue un latent "fail open" : un futur consumer qui oublie le guard DB pourrait activer des fonctions sensibles pendant une panne.

Recommandation. Utiliser `SAFE_DEFAULT_POLICY` lors d'une panne et reserver la policy Succumbrae au bootstrap explicite. Renommer celle-ci en `SUCCUMBRAE_BOOTSTRAP_POLICY` pour clarifier son role.

Priorite. P1.

5.12 Reporting secondaire synchrone

Constat. `ReportService.emit` attend chaque reporter. `DiscordForumReporter` effectue un appel reseau Discord. Certaines commandes non differees emettent le report avant de repondre a l'utilisateur.

Impact. Une destination de reporting lente peut retarder la reponse utilisateur, particulierement ironique si l'incident provient deja de Discord.

Recommandation. Garantir un temps borne pour les reporters distants, deferer les interactions qui peuvent devenir longues et faire du logging local le chemin le plus fiable. Une queue n'est pas necessaire pour la V1.

Priorite. P1.

5.13 Autorisation implicite par hierarchie

Constat. `AuthorizationService` accorde `Capability.SAY` a tout role non-managed place au-dessus du role du bot. `SAY_PLAIN` reste refuse sauf au owner, ce qui est une bonne protection.

Impact. Pour /say, le risque est limite. Pour de futurs outils de moderation/IA, une simple reorganisation de la hierarchie Discord pourrait conferer involontairement une capacite sensible.

Recommandation. Garder l'enum de capabilities, mais passer les futures capabilities sensibles sur une liste de grants explicites (role ID/capability) stockee en configuration ou DB.

Priorite. P2 avant l'ajout d'outils sensibles; P0 avant de donner des actions destructrices a un agent IA.

5.14 /say et mentions Discord

Constat. SayService transmet directement le message a channel.send. Aucune politique allowed_mentions explicite n'est visible.

Impact. Un utilisateur autorise peut potentiellement provoquer des mentions de masse si les permissions du bot le permettent.

Recommandation. Definir une politique de mentions explicite. L'autorisation d'un @everyone, d'un role ou d'un utilisateur doit etre un choix de capability, pas un effet secondaire du texte.

Priorite. P1 faible cout.

6. Refonte des noms trop specifiques

Le but n'est pas d'effacer le metier. Noctis represente un vrai workflow specifique et peut rester nomme ainsi. Le nom Claviger est le nom du produit et n'a pas a etre genericise.

Element actuel	Probleme	Proposition	Pourquoi
SUCCUMBRAE_F ALLBACK_POLI CY	Semantique de fallback + activation sensible	SUCCUMBRAE_B OOTSTRAP_POLI CY puis isolation application	C'est une configuration initiale du serveur, pas un fallback de securite.
AI_OPTION_ROL E_NAME = "option-ia"	Nom serveur fixe dans le core	Champ de policy ou config de feature	Permet a un autre serveur de reutiliser le workflow.
AdultAccessWor kflowService.BA SE_PREFIX/ AI_PREFIX	Convention Noctis codee en dur	Config de feature si reutilisation voulue	Ne pas genericiser si Noctis reste volontairement specifique.
Noctis*	Nom metier	Conserver	Le comportement

Element actuel	Probleme	Proposition	Pourquoi
	specifique		adulte + variantes IA est reellement specifique.
ClavigerBot	Nom produit	Conserver	Nom de l'application, pas une fuite Succumbrae.
salutations_chan nel_name	Nom latin du serveur via policy	Conserver la cle generique; valeur dans bootstrap Succumbrae	La structure est deja correctement decouplee de la valeur.

7. Normalisation de la nomenclature

7.1 Convention cible

La convention souhaitee est raisonnable tant qu'elle ne devient pas mecanique. Les fichiers situes dans un dossier deja tres explicite (database/connection.py) peuvent rester courts si le contexte suffit. En revanche, dans services/, commands/ et models/, les suffixes reduisent reellement l'ambiguite.

7.2 Renommages recommandes

Nom actuel	Nom recommande	Type	Justification
commands/ guild.py	guild_command.py	command	Harmoniser avec member_command.py.
commands/ roles.py	roles_command.py	command	Meme raison.
commands/ report.py	report_command.py	command	Meme raison.
commands/ say.py	say_command.py	command	Eviter collision conceptuelle avec service.
commands/ claviger_command.py	claviger_group_command.py	command group	Le fichier assemble un groupe, pas une commande unique.

Nom actuel	Nom recommande	Type	Justification
services/authorization.py	authorization_service.py	service	Alignement classe/fichier.
services/role_discovery.py	role_discovery_service.py	service	Alignement et lisibilite.
services/role_classifier.py	role_classifier_service.py	service	Classifier utilise comme service applicatif.
services/say.py	say_service.py	service	Alignement avec SayService.
services/guild_policy_bootstrap.py	guild_policy_bootstrap_service.py	service	Convention deja majoritaire.
services/catalog_registry_service.py	catalog_registry.py	registry	La classe est CatalogRegistry, pas ...Service.
services/catalog_sync_planner_service.py	catalog_sync_planner.py	planner	La classe est CatalogSyncPlanner; le suffixe service est artificiel.
models/adult_access.py	adult_access_model.py	model	Eviter ambiguite avec service/feature.
models/member_interest.py	member_interest_model.py	model	Meme raison.
reporting/event.py	report_event.py	model/event	Optionnel; le dossier reporting rend event.py deja assez clair.
database/connection.py	Conserver	infrastructure	Le dossier fournit deja le contexte.
database/schema.py	Conserver	infrastructure	database_schema.py serait redondant.
policies/guild_policy.py	Conserver	policy	Le suffixe policy est deja present.

7.3 Migration

1. Supprimer la duplication de `commands/__init__.py`.
2. Renommer d'abord les fichiers a ambiguite forte (say, roles, authorization).
3. Mettre a jour imports et tests dans le meme commit.
4. Ne pas combiner cette migration avec un refactoring fonctionnel important.
5. Appliquer le formatage/lint puis executer toute la suite.

8. Organisation recommandee des tests

8.1 Etat actuel

La suite contient 252 tests et plus de lignes de test que de code applicatif. C'est un signal positif, mais l'arborescence montre l'histoire du projet : une partie a ete rangee par domaine en fin de V1 et `tests/test_claviger/` conserve les tests issus du decoupage d'un ancien module devenu trop volumineux.

Ce dossier historique n'est donc pas considere comme une erreur d'architecture. La cible doit simplement rendre sa nature evidente ou absorber progressivement ses tests dans les modules actuels.

8.2 Arborescence cible

```
tests/  
|- unit/  
|  |- models/  
|  |- policies/  
|  |- services/  
|  ` - reporting/  
|- integration/  
|  |- database/  
|  ` - repositories/  
|- component/  
|  ` - discord/  
|     |- commands/  
|     ` - ui/  
|- smoke/  
|  |- test_package.py  
|  |- test_main.py  
|  ` - test_bot_composition.py  
|- fixtures/  
|  ` - discord_fakes.py  
`- confest.py
```

Les tests `test_claviger/test_catalog.py`, `test_database.py`, `test_guild.py`, `test_report.py`, `test_roles.py` peuvent rejoindre progressivement `component/discord/commands/admin/`. `helpers.py` peut devenir une fixture partagée plutôt qu'un marqueur historique.

8.3 Gaps prioritaires

1. Echec d'un appel Discord **au milieu** d'un executor et vérification du rapport d'état partiel.
2. `Role managed=True` dans `RoleChannelDiscoveryService`.
3. Deux rôles produisant la même `catalog_key`.
4. Doublons du rôle membre/adulte configuré.
5. Doublons de salons portant le même nom si la résolution par nom reste temporairement utilisée.
6. Configuration de logging runtime et préservation d'un traceback.
7. Rédaction de champs sensibles dans les logs/reports.
8. Erreur du `DiscordForumReporter` lente ou indisponible sans blocage excessif de l'interaction.
9. Valeurs de configuration invalides (`DISCORD_GUILD_ID`, `ADMIN_REPORT_FORUM_ID`).
10. Concurrence de deux soumissions simultanées pour le même membre si le serveur devient plus actif.

9. Architecture cible

9.1 Ne pas reorganiser tout le package immédiatement

La structure actuelle par couches reste suffisamment claire pour la V1. Un grand déplacement de dossiers avant déploiement augmenterait le risque sans bénéfice fonctionnel.

9.2 Refactoring intermédiaire recommande

État actuel

- > fiabiliser erreurs/logs
- > corriger invariants de rôles/catalogues
- > unifier executor/plan/résultat
- > normaliser noms et tests
- > extraire petites factories de composition

9.3 Cible à moyen terme si le projet grossit

`claviger/`

```
| - core/  
| | - role_management/      # plan/résultat/exécution générique  
| | - catalog/              # sync modèles + planner  
| | - authorization/        # capacités et policy de grants  
| ` - reporting/            # contrats d'événements
```

```

|- features/
|  |- member/
|  `-- noctis/          # conserve le metier specifique
|- integrations/
|  |- discord/          # commands, ui, gateways Discord
|  `-- sqlite/          # repositories, migrations
|- applications/
|  `-- succumbrae/      # bootstrap/config initiale du serveur
`-- bootstrap/          # settings + composition du bot

```

Cette cible n'est justifiee que lorsque plusieurs surfaces (Discord + Web + agent) partagent les memes cas d'usage. Avant cela, le decoupage actuel est plus simple.

10. Backlog priorise

P0 - A corriger avant mise en production

- Configurer le logging runtime vers stdout avec niveaux et contexte.
- Conserver les tracebacks complets des erreurs inattendues.
- Brancher /membre, les modales member/noctis et les UI catalogue sur un reporting uniforme.
- Corriger la determination role_manageable afin d'exclure roles managed/default partout.
- Rendre les echecs partiels de mutation Discord observables et testables.
- Verrouiller une version reproductible des dependances pour l'image de deployment.
- Pour Docker : persister la DB dans un volume et valider une procedure de backup/restauration avant mise en service.

P1 - Important immediatement apres le premier deployment

- Detecter/refuser les cle logiques de catalogue dupliquees.
- Renforcer l'identite des roles/salons fixes avec IDs ou resolution unique fail-closed.
- Faire du fallback de panne une policy restrictive; reserver Succumbrae au bootstrap.
- Ajouter une politique explicite allowed_mentions pour /say.
- Ajouter CI : installation, ruff, pytest.
- Ajouter redaction/sanitization des details de reports.
- Borner les reporters reseau.
- Supprimer la duplication de commands/__init__.py.

P2 - Refactoring structurant

- Unifier RoleMutationPlan, RoleMutationResult et executor generique.
- Supprimer la duplication is_publicly_ready.
- Normaliser la nomenclature fichiers/classes.
- Reorganiser completement la suite de tests.

- Centraliser les guards Discord recurrents (guild/owner/error boundary).
- Charger la configuration dans un objet Settings valide une seule fois.

P3 - Amelioration a moyen terme

- Extraire le wiring en modules de composition.
- Introduire quelques Protocols/gateways autour de Discord et des repositories uniquement lorsqu'une seconde interface en a besoin.
- Ajouter lock applicatif par (guild_id, member_id) si les soumissions concurrentes deviennent plausibles.
- Ajouter des metriques legeres si les logs ne suffisent plus.

P4 - Evolutions futures

- Multi-guild reel et configuration dynamique.
- Interface Web d'administration.
- Outils de moderation avec capability grants explicites.
- Tools IA avec schemas d'action, audit et confirmation.
- Agent conversationnel.
- Sentry ou centralisation de logs seulement si le volume d'incidents le justifie.

11. Lecture du projet par un recruteur technique

11.1 Ce que le projet demontre reellement

Un Senior Backend ou Tech Lead peut constater :

- conception d'un workflow idempotent/reconciliant;
- separation planification/execution;
- persistance asynchrone SQLite et migrations;
- transactions et rollback;
- injection de dependances sans framework;
- gestion de politiques multi-guild partielle;
- models immuables largement utilises;
- tests unitaires, integration SQLite et tests de composants Discord;
- prise en compte de permissions Discord et hierarchie de roles;
- premiere architecture d'observabilite multi-sink;
- capacite a refactorer un module devenu trop volumineux plutot qu'a l'abandonner.

Ce sont des sujets Backend defendables, independamment du fait qu'il s'agisse d'un bot Discord.

11.2 Ce qui donne une bonne impression

Le meilleur signal n'est pas le nombre de fichiers. C'est la presence de choix dont la raison est visible : planner pur, preflight, transaction multi-catalogue, conservation de metadata, safe default, tests des cas limites.

Le README mis a jour explique correctement la reconciliation et annonce 252 tests; le comptage statique de l'audit retrouve bien 252 tests.

11.3 Ce qui peut faire amateur aujourd'hui

- print au demarrage alors qu'une infrastructure de reporting existe deja;
- erreurs silencieuses dans certaines modales;
- duplication de fichier dans commands/__init__.py;
- conventions de nommage a moitie migrees;
- pas de CI/dependency lock/deploiement dans la branche auditee;
- exemples de fixtures de test tres explicites lies au contenu adulte.

Sur ce dernier point, il ne s'agit pas d'un probleme technique. Mais pour un repository vitrine public, des fixtures neutres (theme-a, theme-b) eviteraient que le domaine adulte detourne l'attention de l'architecture. Le README a deja adopte ce ton neutre.

11.4 Gain de credibilite maximal pour un cout raisonnable

1. Faire passer le reporting/logging au niveau production.
2. Ajouter CI + lock de dependances.
3. Corriger les deux ou trois invariants de role/catalogue identifies.
4. Finir le rangement/nomenclature des tests et fichiers.
5. Documenter une architecture decision courte sur le choix reconciliation/planner/executor.
6. Montrer un deployment Docker reproductible avec volume/backup SQLite.

Ces elements augmenteront plus la credibilite qu'un ajout premature de microservices, Redis ou Kubernetes.

11.5 Questions d'entretien auxquelles l'auteur doit pouvoir repondre

- Pourquoi reconstruire l'etat au submit plutot que faire confiance au modal ?
- Quelle difference entre preflight atomique et execution réellement atomique sur une API externe ?
- Pourquoi utiliser SQLite ici, et a partir de quand migrer vers PostgreSQL ?
- Pourquoi un planner pur facilite-t-il les tests ?
- Quelle donnee appartient a Discord, laquelle appartient a Claviger ?
- Comment le catalog sync evite-t-il un demi-commit entre deux catalogues ?
- Pourquoi le ReportService utilise-t-il un Protocol ?
- Comment transformer le systeme de capabilities pour un agent IA sans lui donner des privileges implicites ?
- Pourquoi une Clean Architecture complete serait-elle probablement excessive aujourd'hui ?

12. Dette technique

Dette acceptable

- ClavigerBot comme composition root explicite unique tant que le nombre de modules reste actuel.
- SQLite pour une instance mono-processus sur mini-PC.
- Resolution par noms sur la V1 si le serveur est strictement controle et les doublons preflightes.
- Report Discord comme canal d'administration secondaire.

Dette a traiter

- nomenclature incomplete;
- organisation tests en migration;
- duplications executor/models;
- configuration eparpillee;
- absence de CI/lock;
- dependances concretes Discord dans les services qui devront etre reutilises ailleurs.

Dette dangereuse

- erreurs inattendues sans trace exploitable;
- mutation Discord partielle non documentee/non reportee;
- incoherence de role_manageable;
- futurs outils sensibles utilisant la confiance implicite par hierarchie;
- futur consumer utilisant la policy Succumbrae active pendant une panne DB sans guard fail-closed.

Conclusion

Claviger a un potentiel professionnel reel. La V1 montre deja des competences Backend que beaucoup de projets personnels n'exposent pas : orchestration, transactions, modelisation d'etat, testabilite et traitement des permissions.

Le projet n'a pas besoin d'une architecture plus impressionnante. Il a besoin que sa **fiabilite operationnelle atteigne le niveau de son coeur metier**. Si les PO/P1 sont traitees proprement puis expliques dans le README et le deploiement, Claviger devient une piece de portfolio credible pour un profil Backend en evolution vers AI Engineer.